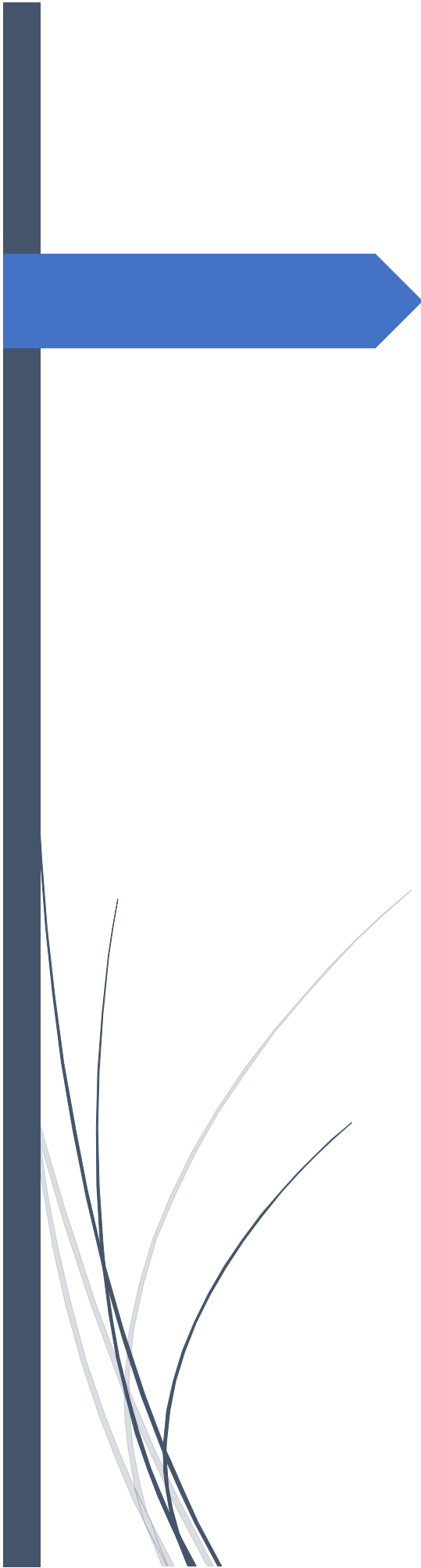




Smart EV Charging Hub with V2G & Renewable Scheduling

Contents

List of figures	2
List of tables	3
List of Graphs	3
INTRODUCTION.....	4
1. Petri Net model using PIPE2	6
1.1 Model Description	6
1.2 Model Simulation	10
1.3 Model Analysis.....	14
1.3.1 Petri net incidence and marking.....	14
1.3.2 Reachability/coverability graph	15
1.3.3 Invariants of the system	15
1.3.4 Syphons and traps	16
1.3.5 Implications and their reasoning.....	16
1.3.6 Simulation and results.....	17
2. Discrete Event Simulation in Anylogic.....	18
2.1 Model description.....	18
2.2 Model simulation	21
2.3 Model Results and Analysis	22
3. Optimization	26
4. Conclusions.....	28
References	29

List of figures

Figure 1 - Facilitating energy efficiency by integrating a higher proportion of renewable energy .	4
Figure 2- Vehicle-to-Grid feature.....	5
Figure 3- Petri Net model of the smart charging	6
Figure 4 – Transitions’ configuration (1).....	8
Figure 5 – Transitions’ configuration (2).....	8
Figure 6 – Places’ configuration.....	9
Figure 7 – Initial State before any firing occurs	10
Figure 8 – T_Arrival fires, and T_Slow_PV & T_Fast_PV are ready to be chosen and fire	10
Figure 9 – Depending on our choice the vehicles arrive on the charge places (we did 2 firings here for 2 vehicles, just to show how the tokens relocate)	11
Figure 10 – After charging for the specified times they leave the system, we had one vehicle in the fast hub and one in the slow one, that is why we have 2 tokens on the P_Charged	11
Figure 11 – If we chose to fire T_V2G then we not only give back to the P-Grid (value of tokens increased), but we also go out of the system	12
Figure 12 – T_Arrival fires, and T_Slow_Battery & T_Fast_Battery are ready to be chosen and fire (the other part remains the same as we explained).....	12
Figure 13 – T_Arrival fires, and T_Slow_Grid & T_Fast_Grid are ready to be chosen and fire	13
Figure 14 – System after 5000 simulations	13
Figure 15 – Incidence matrices.....	14
Figure 16 – Initial marking.....	14
Figure 17 – Simulation results	18
Figure 18 – DES model in Anylogic	18
Figure 19 – SelectOutputs’ configuration	19
Figure 20 – Queues’ configuration	19
Figure 21 – Other blocks’ configuration	20
Figure 22 – Recharging event	20
Figure 23 – Anylogic model simulation.....	21
Figure 24 – System while running	24
Figure 25 – Some statistics’ configurations	25
Figure 26 - Overview	27
Figure 27 – Importance of scheduling	27

List of tables

Table 1- Places	6
Table 2 – Transitions	7
Table 3 - Results.....	23
Table 4 – Resource pool utilization	24

List of Graphs

EV_Arrival.....	20
EV_Arrival vs Sink.....	20
PV_Levels.....	20
Distribution Graphs.....	23

INTRODUCTION

As the electric-vehicle (EV) market continues to skyrocket, so will our reliance on electrical energy. Today, there is already enough generation capacity to meet future demand, but renewable energy isn't always produced when and where it's needed. What if we could balance the grid throughout the day, boost energy efficiency, and accommodate a high share of renewables?

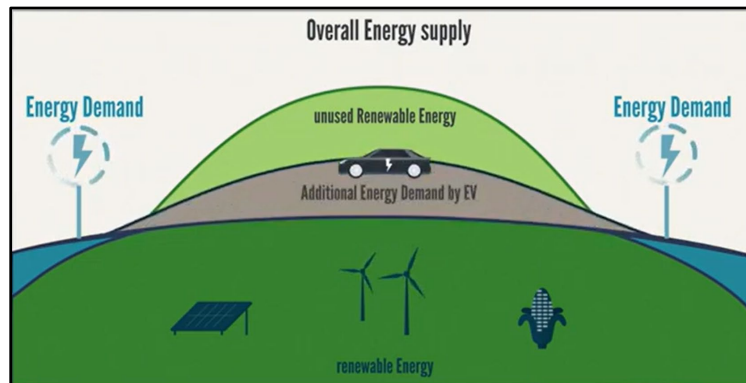


Figure 1 - Facilitating energy efficiency by integrating a higher proportion of renewable energy

Within the charging community, that's exactly what the game-changing new feature of the Combined Charging System (CCS) **Smart Charging** enables. Smart Charging gives you full control of your charging session and unlocks bi-directional power flow (Vehicle-to-Grid, or V2G). On technical, ecological and economic levels, it delivers multiple benefits:

- **Optimized charging schedules**
From the comfort of home, you can remotely schedule your charging session to run during off-peak hours, relieving grid stress and taking advantage of inexpensive, renewable energy as it becomes available.
- **Bi-directional power flow (V2G)**
Your EV becomes a mobile power bank, able to feed electricity back into the grid, or even directly into a home or building, transforming vehicles into distributed energy resources.

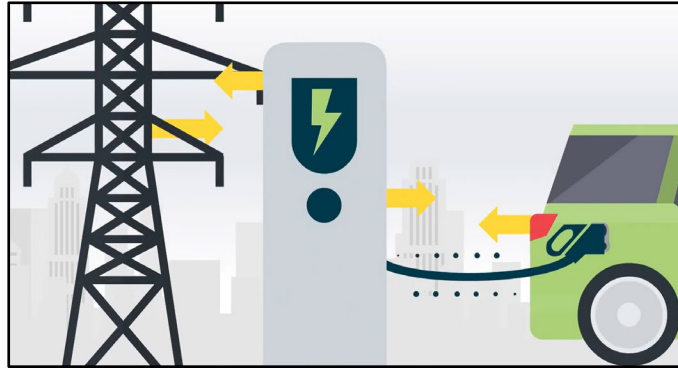


Figure 2- Vehicle-to-Grid feature

- **Extended grid resilience & deferred infrastructure costs**
A fleet of V2G-enabled EVs forms a “virtual power plant,” balancing supply and demand in real time and reducing the need for new plants or costly grid upgrades.
- **Lower energy bills & total cost of ownership**
By shifting usage to off-peak hours and participating in energy markets, EV owners can cut electricity bills and recoup savings over the vehicle’s lifetime.

In the following section of this project, we will present a Vehicle-to-Grid (V2G) scenario using Petri Nets. Through this modeling, we will uncover the system’s benefits, assess the queue delays in vehicle charging, and then elaborate on the results via dynamic simulation in AnyLogic, providing a comprehensive overview of the smart charging performance.

1. Petri Net model using PIPE2

1.1 Model Description

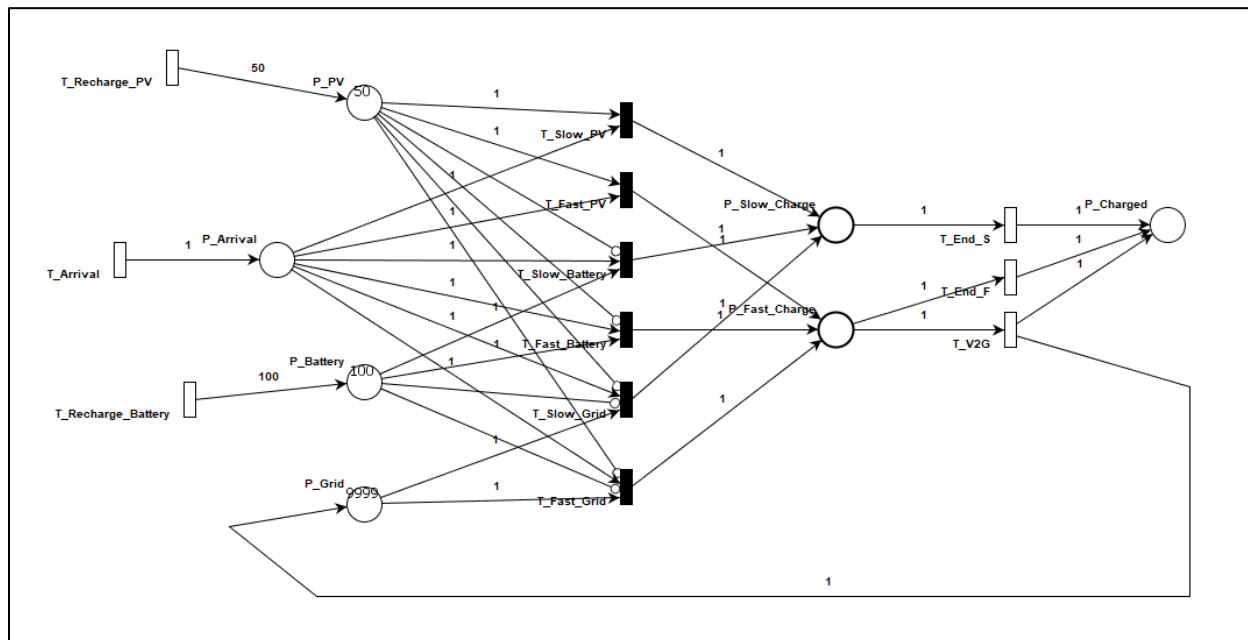


Figure 3- Petri Net model of the smart charging

The Petri Net in Figure 3 represents our Smart EV Charging Hub with V2G and renewables. We have 12 places and 9 transitions:

Places	Description
P_Arrival	EV arriving at the charge station
P_PV	PV Energy
P_Grid	Energy drawn from grid
P_Battery	Stored battery energy
P_Charge_Slow	EV in slow-charge hub
P_Charge_Fast	EV in fast-charge hub
P_Charged	EV ready to leave the station

Table 1- Places

Transitions	Description
T_Arrived	EV arrival event
T_Slow_PV	Choose slow charging through PV
T_Fast_PV	Choose fast charging through PV
T_Slow_Battery	Choose slow charging through Battery
T_Fast_Battery	Choose slow charging through Battery
T_Slow_Grid	Choose slow charging through Grid
T_Fast_Grid	Choose slow charging through Grid
T_End_S	Slow-charge completion
T_End_F	Fast-charge completion
T_V2G	V2G (departure to grid)
T_Recharge_PV	Gives the same value as in the beginning to P_PV
T_Recharge_Battery	Gives the same value as in the beginning to P_Battery

Table 2 – Transitions

Tokens represent “energy units” or “EVs” in various states. Places model the available energy sources, the EVs in different charging states, and the finished/voiced-back states. Transitions fire when an event occurs (an EV arrives, charging completes, or V2G export happens), moving tokens between places.

In our Petri-net model of the Smart EV Charging Hub, we represent each component of the charging ecosystem, renewable PV panels, a stationary battery, and the utility grid, as separate “supply” places (P_PV, P_Battery, P_Grid). EVs arrive according to the timed transition T_Arrival (rate = 5 vehicles per time unit) and carry a single token into place P_Arrived. From there, a set of immediate transitions (T_Slow_PV, T_Fast_PV, T_Slow_Battery, T_Fast_Battery, T_Slow_Grid, and T_Fast_Grid) split each EV’s charging request into three possible energy sources and two charging speeds. We assigned each of these immediate transitions equal weight so that, in the absence of other controls, vehicles are offered the choice of slow or fast charging from PV, battery, or grid uniformly.

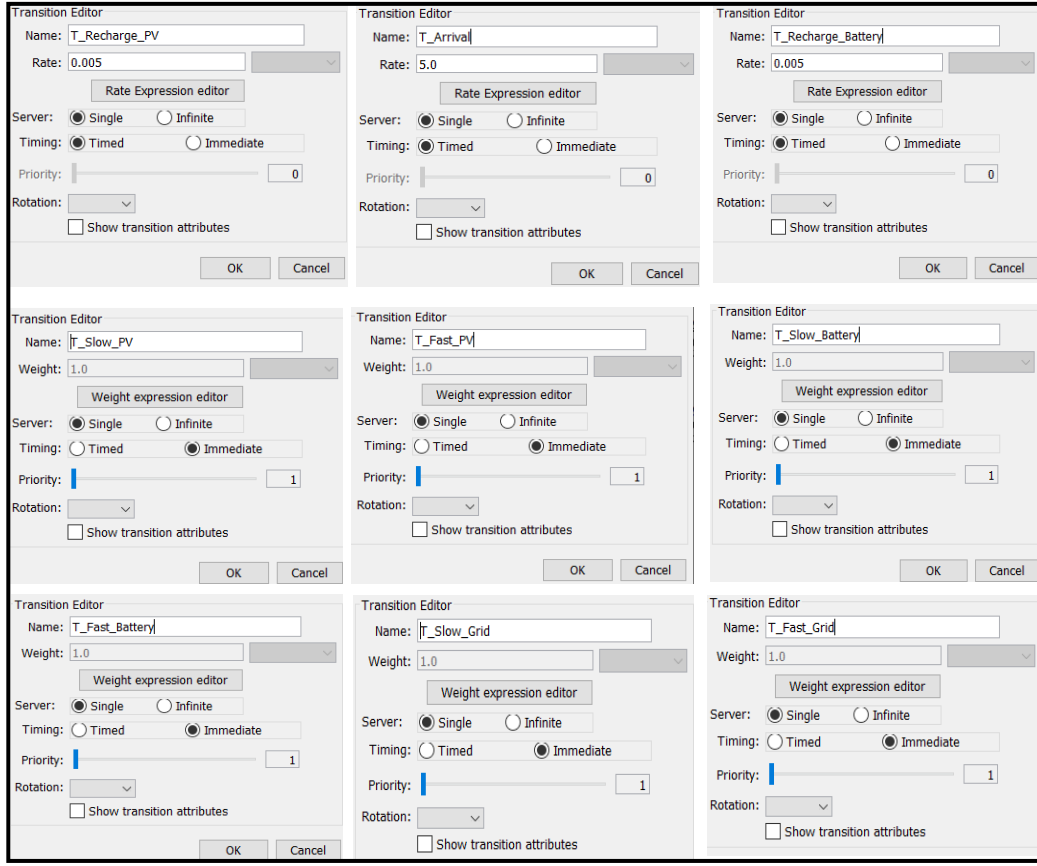


Figure 4 – Transitions' configuration (1)

Once an energy source–speed pair is chosen, the EV's token moves into either P_Slow_Charge or P_Fast_Charge, each of which has a capacity of ten tokens, modeling the ten slow bays and ten fast bays available at the hub. The timed transitions T_End_S and T_End_F (rate = 0.5 and 2.0, respectively) then simulate charging durations: at a rate of 0.5, the mean service time in the slow charger is two-time units, while at a rate of 2.0, the fast charger completes in an average of 0.5-time units. Upon service completion, tokens flow through T_End_S or T_End_F into P_Charged, signifying a finished charge, and then exit the net via the departure place.

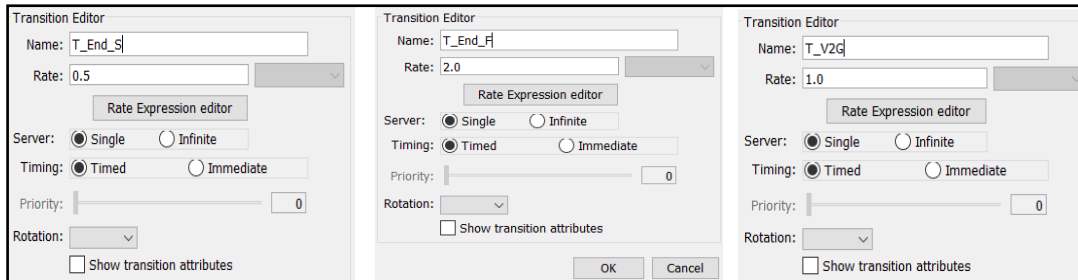


Figure 5 – Transitions' configuration (2)

To capture vehicle-to-grid (V2G) discharge, we included a timed transition T_V2G (rate = 1.0) that draws a token from $P_Charged$ back to a P_Grid_Sell place, representing the energy returning to the grid. We also needed the system to replenish its renewable and battery stocks periodically, so we modeled two additional timed transitions, $T_Recharge_PV$ and $T_Recharge_Battery$ (both with rate = 0.005), each feeding tokens back into P_PV and $P_Battery$ at low frequency, roughly every 20-time units. By carefully choosing rates, priorities, and place capacities, the Petri net cleanly captures the dynamic interplay of arrivals, resource allocation, service, and energy flows (both charging and discharging) in our Smart EV Charging Hub.

Figure 6 – Places’ configuration

In our design, the little “bar” symbols on each PV- and battery-fed transition do double duty as inhibitor arcs and as an implicit prioritization mechanism. Because slow- and fast-charge branches from P_PV and $P_Battery$ are immediate transitions (with higher firing priority than any timed or grid branches), the net will always draw from the solar panels or the station battery first, so long as those places still hold tokens. Only when a renewable source is empty (i.e. its token count has dropped to zero and thus its inhibitor arc blocks its outgoing transitions) will the grid-powered paths become enabled. In this way, inhibitors neatly encode “use green power first, fall back to the grid only when nothing else remains” without any extra scheduling logic or priority sliders.

Likewise, we deliberately kept every branch weight at exactly one and assigned exactly one token per place, not the real kWh values, simply to illustrate the flow of “energy units” through the system. By treating each token as an abstract “quantity of charge” rather than tying P_PV or $P_Battery$ to actual kilowatt-hours, we avoid cluttering the net with arithmetic expressions or fractional token semantics. In a real deployment one would scale those weights and place capacities to match, say, 150 kWh of storage or 50 kWh of solar output, but here unity weights keep the focus on how tokens move, inhibit, and queue, proving the structure of our Smart EV-Charging Hub is correct before adding the numerical realism of true energy accounting.

1.2 Model Simulation

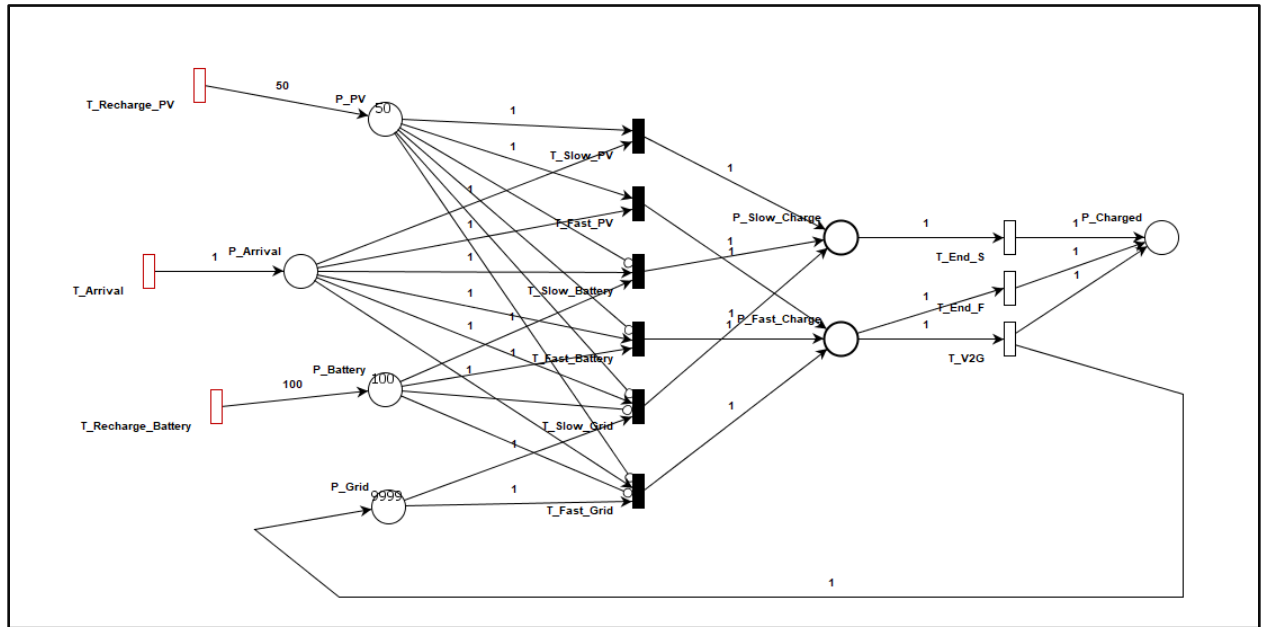


Figure 7 – Initial State before any firing occurs

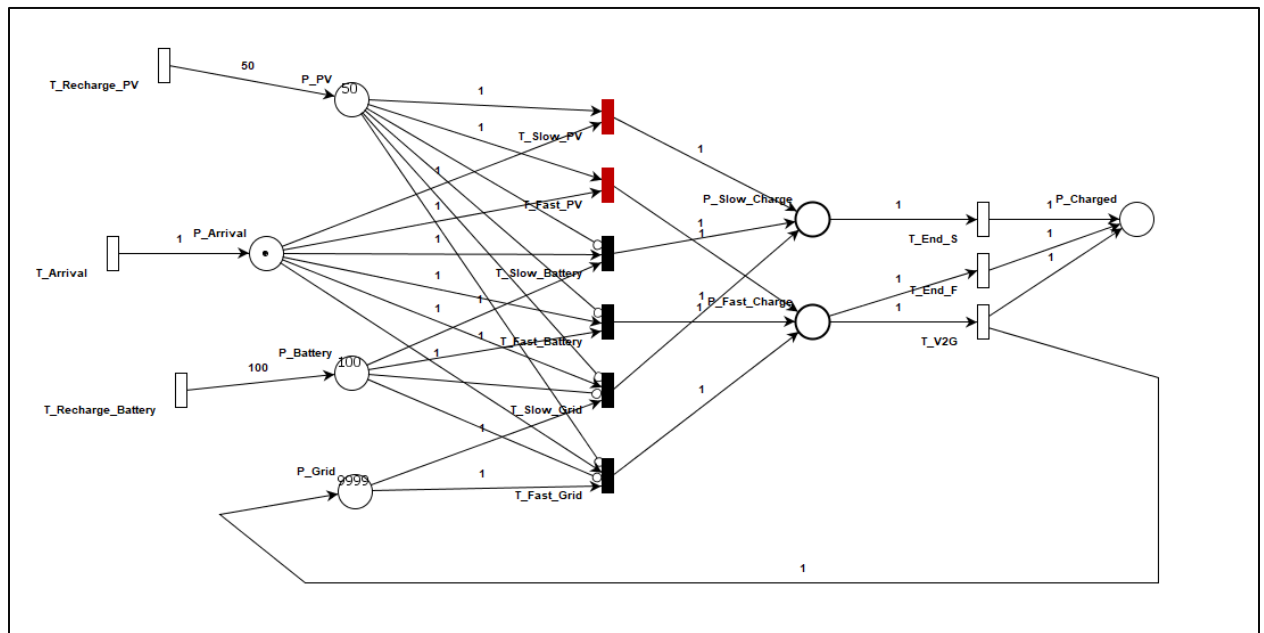


Figure 8 – $T_Arrival$ fires, and T_Slow_PV & T_Fast_PV are ready to be chosen and fire

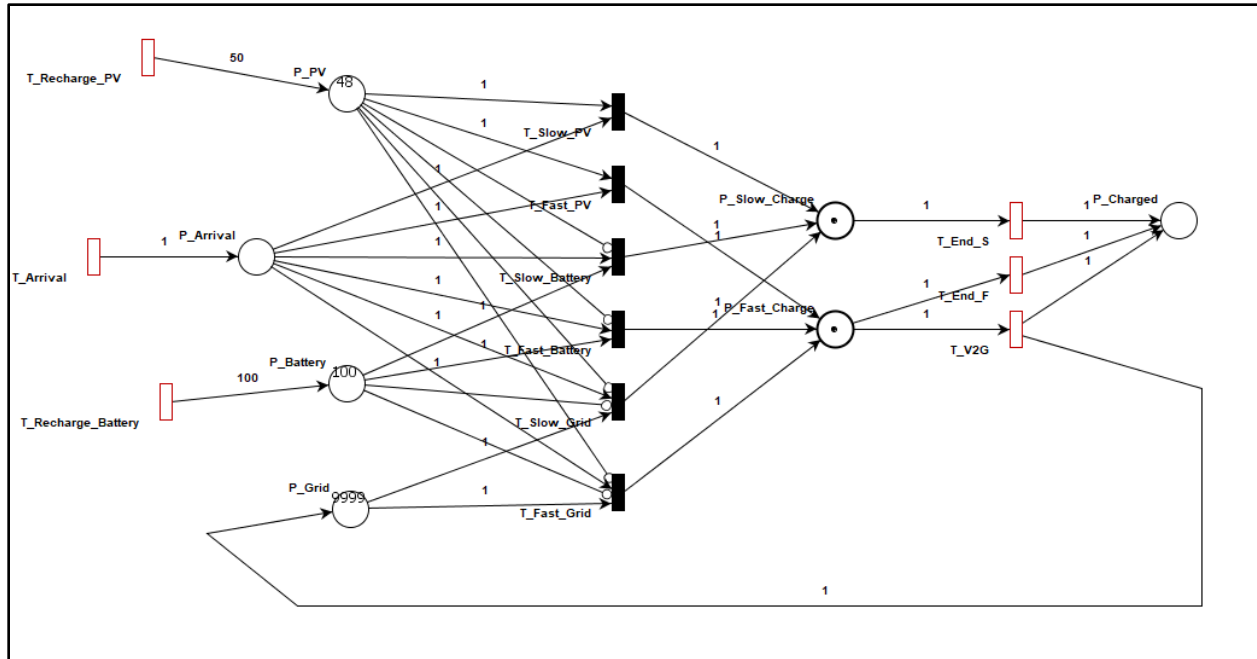


Figure 9 – Depending on our choice the vehicles arrive on the charge places (we did 2 firings here for 2 vehicles, just to show how the tokens relocate)

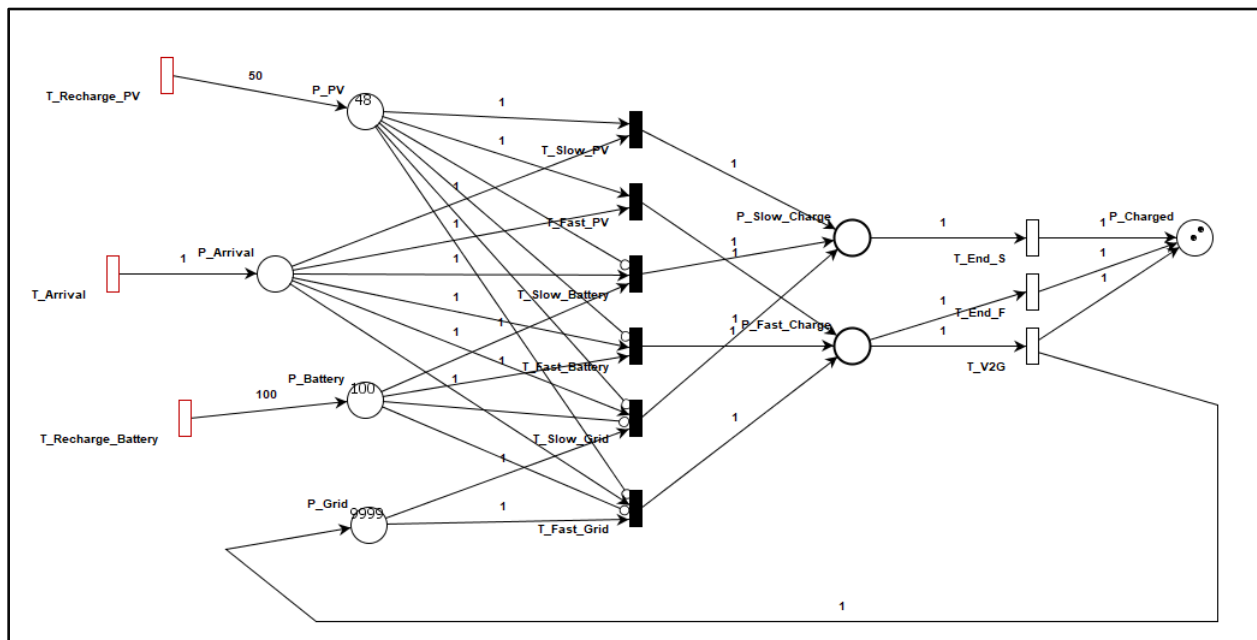


Figure 10 – After charging for the specified times they leave the system, we had one vehicle in the fast hub and one in the slow one, that is why we have 2 tokens on the P_Charged

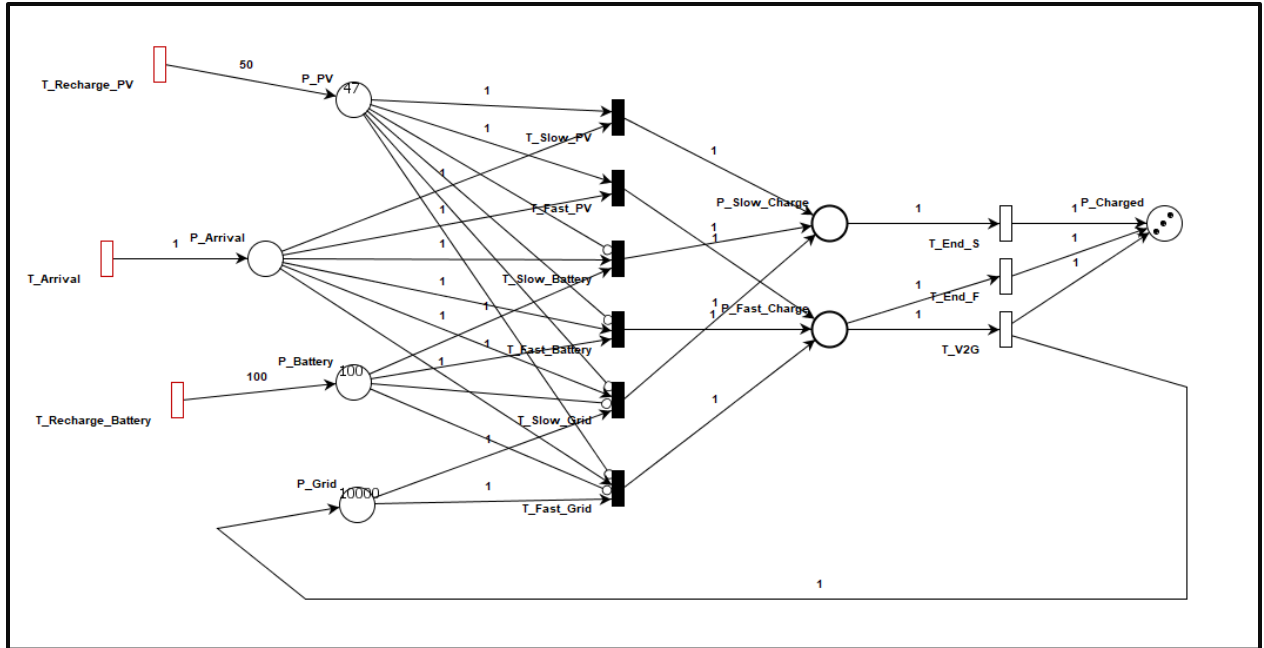


Figure 11 – If we chose to fire T_{V2G} then we not only give back to the P_{Grid} (value of tokens increased), but we also go out of the system

Up to now we did 3 simulations in the system only charging using PV. As we explained above at the moment we do not have any more tokens in P_{PV} , so we are going to use for charging $P_{Battery}$, as below.

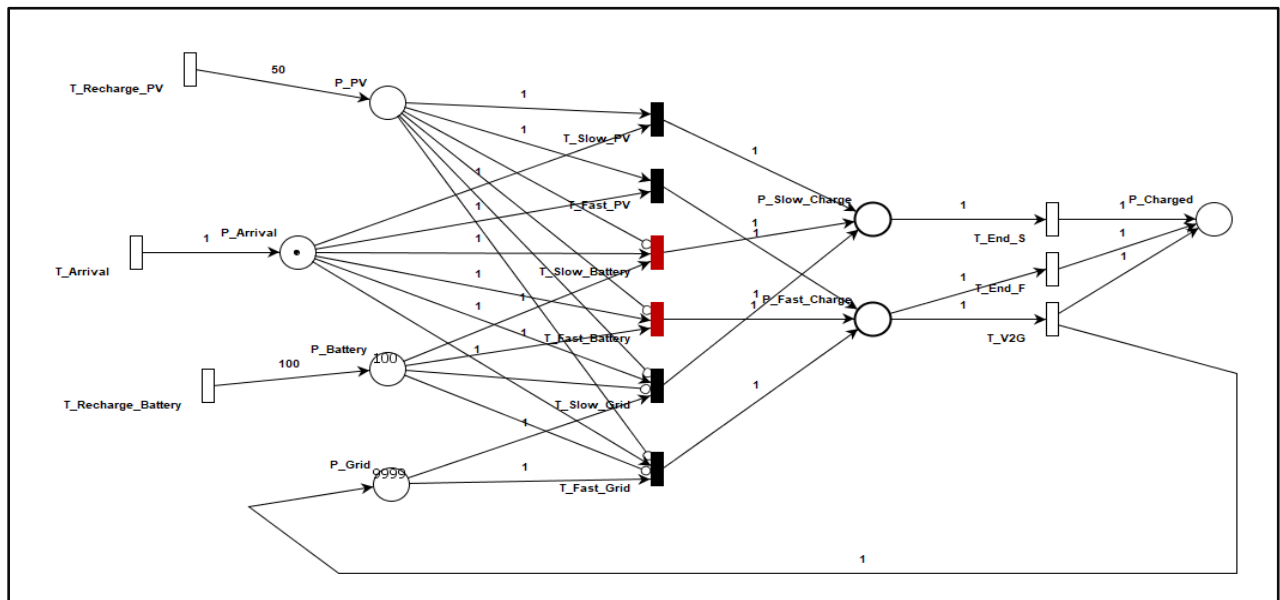


Figure 12 – $T_{Arrival}$ fires, and $T_{Slow_Battery}$ & $T_{Fast_Battery}$ are ready to be chosen and fire (the other part remains the same as we explained)

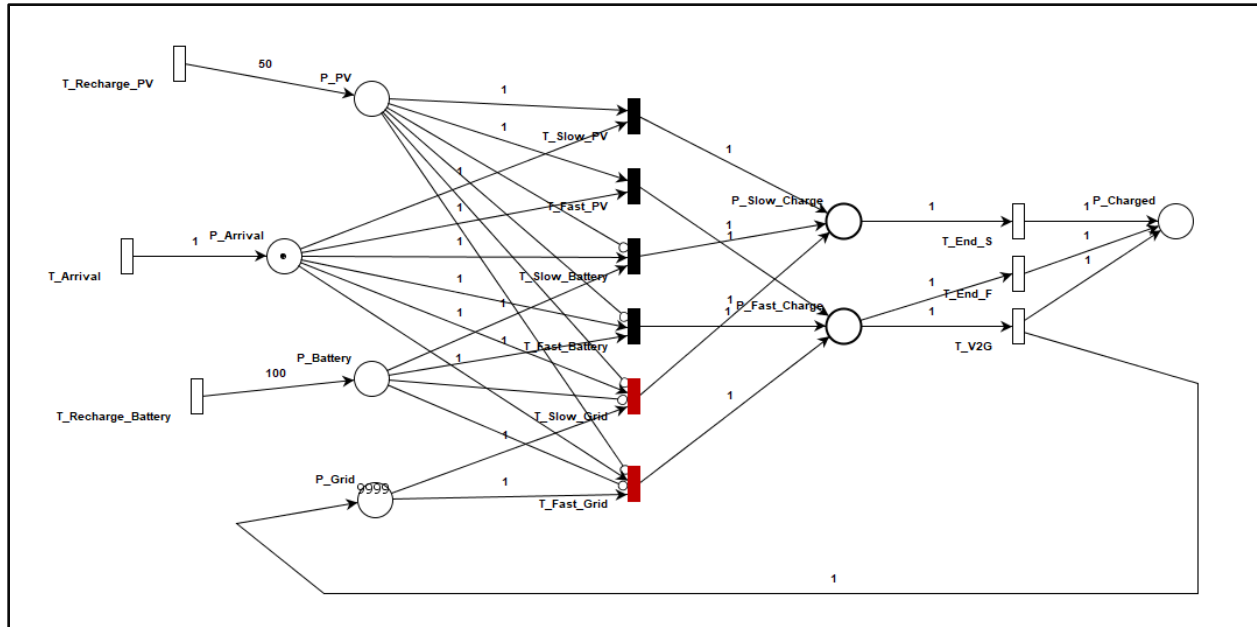


Figure 13 – $T_Arrival$ fires, and T_Slow_Grid & T_Fast_Grid are ready to be chosen and fire

When you subject our Petri-net to a long series of firings, effectively simulating a busy day at the charging hub, we begin to see exactly the same congestion patterns that we will later observe in AnyLogic. As $T_Arrival$ fires repeatedly, tokens flood into $P_Arrived$ faster than they can be consumed whenever the downstream charging places are full. In the net, $P_Arrived$ acts as a waiting room: its marking climbs steadily, and the incidence of multiple tokens there is our “queue length.” Because the three pairs of immediate transitions out of $P_Arrived$ (slow-PV, fast-PV, slow-battery, etc.) can only fire when their source place still holds tokens and because the timed transitions T_End_S and T_End_F each return exactly one token to $P_Charged$ at finite rates, we will see periods during which $P_Arrived$ accumulates dozens or even hundreds of tokens, exactly mirroring growing wait times at the station entrance.

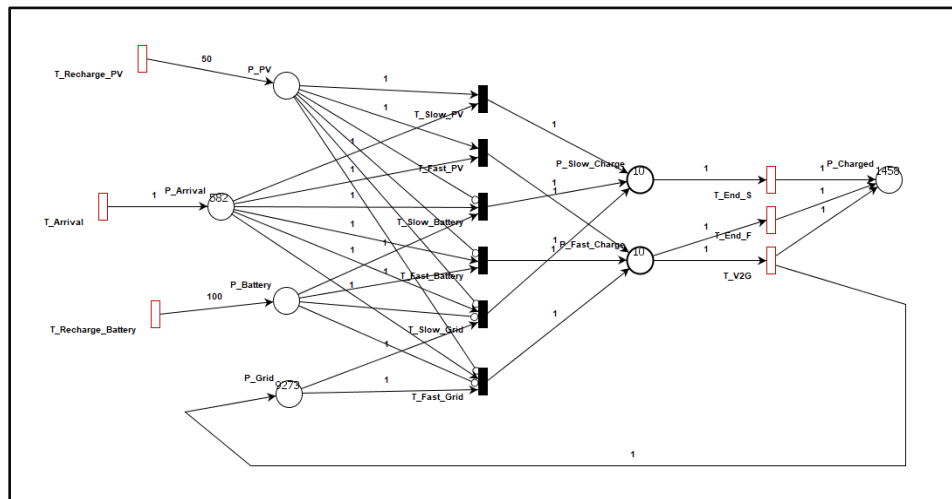


Figure 14 – System after 5000 simulations

Meanwhile, the capacity restrictions on P_Slow_Charge and P_Fast_Charge(their upper limit of ten tokens) introduce hard blockages in the net. Whenever either place is “saturated,” its inhibitor arcs prevent any more tokens from entering that branch, so the corresponding immediate transitions simply cannot fire. That in turn reinforces the buildup at P_Arrived, because no new service can be started until a slot frees up. Simulating many replications of this timed stochastic net will produce statistics on how often those capacities bind, how long tokens dwell in the arrival place, and how frequently the PV and battery-first inhibitor logic actually deflects load onto the grid branch.

1.3 Model Analysis

1.3.1 Petri net incidence and marking

Using PIPE2 we are able to generate the forward, backward and combined incidence matrices. They show resource-consumption logic.

Forwards incidence matrix f^+													
	T_Arrival	T_End_FT	T_End_ST	T_Fast_Battery	T_Fast_Grid	T_Fast_PVT	T_Slow_Battery	T_Slow_Grid	T_Slow_PVT	T_V2G	T_Recharge_PVT	T_Recharge_Battery	
P_Arrival	1	0	0	0	0	0	0	0	0	0	0	0	
P_Battery	0	0	0	0	0	0	0	0	0	0	0	100	
P_Charged	0	1	1	0	0	0	0	0	0	1	0	0	
P_Fast_Charge	0	0	0	1	1	1	0	0	0	0	0	0	
P_Grid	0	0	0	0	0	0	0	0	0	1	0	0	
P_PV	0	0	0	0	0	0	0	0	0	0	50	0	
P_Slow_Charge	0	0	0	0	0	0	1	1	1	0	0	0	
Backwards incidence matrix f^-													
	T_Arrival	T_End_FT	T_End_ST	T_Fast_Battery	T_Fast_Grid	T_Fast_PVT	T_Slow_Battery	T_Slow_Grid	T_Slow_PVT	T_V2G	T_Recharge_PVT	T_Recharge_Battery	
P_Arrival	0	0	0	1	1	1	1	1	1	0	0	0	
P_Battery	0	0	0	1	0	0	1	0	0	0	0	0	
P_Charged	0	0	0	0	0	0	0	0	0	0	0	0	
P_Fast_Charge	0	1	0	0	0	0	0	0	0	1	0	0	
P_Grid	0	0	0	0	1	0	0	1	0	0	0	0	
P_PV	0	0	0	0	0	1	0	0	1	0	0	0	
P_Slow_Charge	0	0	1	0	0	0	0	0	0	0	0	0	
Combined incidence matrix f													
	T_Arrival	T_End_FT	T_End_ST	T_Fast_Battery	T_Fast_Grid	T_Fast_PVT	T_Slow_Battery	T_Slow_Grid	T_Slow_PVT	T_V2G	T_Recharge_PVT	T_Recharge_Battery	
P_Arrival	1	0	0	-1	-1	-1	-1	-1	-1	0	0	0	
P_Battery	0	0	0	-1	0	0	-1	0	0	0	0	100	
P_Charged	0	1	1	0	0	0	0	0	0	1	0	0	
P_Fast_Charge	0	-1	0	1	1	1	0	0	0	-1	0	0	
P_Grid	0	0	0	0	-1	0	0	-1	0	1	0	0	
P_PV	0	0	0	0	0	-1	0	0	-1	0	50	0	
P_Slow_Charge	0	0	-1	0	0	0	1	1	1	0	0	0	

Figure 15 – Incidence matrices

The initial marking is as follows:

	P_Arrival	P_Battery	P_Charged	P_Fast_Charge	P_Grid	P_PV	P_Slow_Charge
Initial	0	100	0	0	9999	50	0

Figure 16 – Initial marking

Only T_Arrival, T_Recharge_Battery, and T_Recharge_PV are enabled at the start.

In our model, the combined incidence matrix ($I = I^+ - I^-$) effectively summarizes the net token changes caused by each transition. However, it leads to a partial loss of structural information. Specifically, it does not reveal which tokens are required simultaneously to enable a transition—e.g., transitions like `T_Fast_Battery` require tokens from both `P_Arrival` and `P_Battery`, but this dependency is not evident in the combined matrix. Furthermore, token flow directionality is lost: a net value of 0 may result from simultaneous consumption and production, which is indistinguishable in I . While purely additive transitions like `T_Recharge_Battery` are represented accurately, the combined matrix does not capture the exact logic of enablement conditions, making it unsuitable for full behavioral analysis without reference to the pre (I^-) and post (I^+) matrices.

1.3.2 Reachability/coverability graph

The reachability graph for our Petri net model could not be generated due to the complexity and potential infiniteness of the state space. The system begins with high initial markings in resource places (e.g., `P_Grid` = 9999, `P_Battery` = 100, `P_PV` = 50), and transitions like `T_Arrival` and `T_Recharge_Battery` allow continuous token flow. As a result, the number of reachable markings grows rapidly, and the state space becomes intractable for exhaustive enumeration. This makes it impossible to construct a complete reachability graph using standard analysis tools. Consequently, while simulation confirms proper cyclic behavior and token flow, the lack of a reachability graph means that properties like boundedness, liveness, and deadlock-freeness cannot be conclusively verified through structural reachability analysis alone.

1.3.3 Invariants of the system

The structural analysis reveals that no positive T-invariants were found. T-invariants typically indicate sequences of transitions that return the system to its initial marking, thus providing evidence of cyclic and repeatable behavior. The absence of T-invariants implies that the model cannot be structurally proven to be live, meaning there is no guarantee that all transitions remain potentially fireable across all reachable markings.

Similarly, the analysis shows that the model is not covered by positive P-invariants, which are essential for proving boundedness and resource conservation. A P-invariant represents a linear combination of places where the total token count remains constant, ensuring that resources such as vehicles or energy are neither lost nor accumulated indefinitely. The absence of P-invariants means the model cannot structurally guarantee that places like `P_Battery`, `P_PV`, or `P_Charged` remain within a defined limit across all possible executions. This introduces the risk

of token overflow or resource depletion, particularly under rare or edge-case conditions, while boundedness may still hold in practical runs.

1.3.4 Syphons and traps

A quick check of our net’s inhibitor-augmented structure shows that one obvious trap is formed by the “charged vehicles” place, `P_Charged`. Once a token reaches there, it never leaves again, in other words, that place can only gain tokens and once marked, stays marked. This exactly matches the definition of a trap and reflects the real-world fact that once a car has finished charging, it does not re-enter the charging workflow.

By contrast, a classic siphon would be a set of places that, once emptied, can never be refilled. In our model there is no nontrivial siphon, because every place that can lose tokens is fed by a corresponding recharge transition after a period of time.

1.3.5 Implications and their reasoning

Our Petri net includes “open” replenishment transitions (`T_Recharge_PV` and `T_Recharge_Battery`) together with a virtually inexhaustible grid source (`P_Grid = 9 999`), the usual conservation laws that give rise to P-invariants simply cannot hold. In a closed, conservative net, every token that leaves one place must eventually be deposited somewhere else, so one can write linear combinations of place-markings that remain constant. Here, however, our recharge arcs continuously inject new tokens, while the arrival and V2G transitions eventually remove them, so there is no fixed sum to conserve across all executions. In the same way, the inhibitor arcs we use to give renewables priority break the symmetry of the flow: they enforce one-way blocking conditions that prevent us from finding any nontrivial T-invariant cycles. Put bluntly, any positive invariant (place or transition) would require that the net behave like a perfect closed loop, but ours is deliberately designed with unbounded, one-directional flows to model real-world energy injections and withdrawals.

Because of these modeling choices, the absence of positive P- and T-invariants is not a flaw in PIPE2 or in our analysis, but a direct reflection of the fact that our hub is meant to draw down and then replenish its various energy stores indefinitely. In practice, real stations are of course bounded by hardware capacities and operating schedules: a PV array can only generate so much before sundown, a battery pack only holds a finite kWh, and the grid connection is contractually rate limited. To “protect” against the theoretical unboundedness revealed by the lack of invariants, one simply must encode those real-world limits back into the net. Concretely, we would add capacity restrictions on each place, e.g. set `P_PV`’s capacity to the array’s maximum daily output, `P_Battery` to the pack’s rated capacity, and `P_Grid` to the contractual draw limit and perhaps cap the recharge transition rates so that they cannot fire infinitely fast. Once those bounds are in place, the net regains positive P-invariants (the total energy across all

stores plus in-flight vehicles becomes fixed) and T-invariants reappear (the recharge–discharge cycle closes), guaranteeing that every transition remains potentially fireable only within a finite, safe operating envelope.

1.3.6 Simulation and results

After running the PIPE2 simulation for 10 000 firings across 30 independent replications, we obtain the steady-state average token counts shown above. The most striking outcome is the very large mean backlog at the arrival place, $\langle P_Arrival \rangle \approx 638 (\pm 99.4)$, indicating that vehicles pile up much faster than our chargers can process them. Renewable energy tokens in P_PV now average $\langle P_PV \rangle \approx 2.40 (\pm 2.53)$, a modest increase over zero but still showing that PV power is almost immediately consumed whenever it appears. The storage battery remains lightly loaded on average ($\langle P_Battery \rangle \approx 3.09 \pm 16.8$), suggesting it only occasionally buffers energy when both PV and grid resources are constrained.

Both charging stations sit near their capacity limits: the slow charger holds about $\langle P_Slow_Charge \rangle \approx 9.93 (\pm 0.04)$ tokens and the fast charger about $\langle P_Fast_Charge \rangle \approx 9.65 (\pm 0.08)$ tokens out of their maximum ten, confirming that these resources are almost always fully occupied whenever vehicles arrive. Downstream, the grid resource accumulates an average of $\langle P_Grid \rangle \approx 9\,130 (\pm 248.7)$ tokens, since our V2G departures fail to meaningfully deplete this large supply under current parameters. Finally, the charged-vehicle place grows steadily $\langle P_Charged \rangle \approx 1\,440.7 (\pm 33.0)$, capturing total throughput over the firing horizon. Together, these results show that under the chosen rates and capacities, PV and battery inputs are quickly consumed, chargers remain fully occupied, and a severe queue develops at arrival. If arrival rates stay at these levels, it will be necessary to rebalance energy inputs or expand charging capacity to prevent indefinite queue growth.

Petri net simulation results		
Place	Average number of tokens	95% confidence interval (+/-)
P_Arrival	637.66343	99.38891
P_Battery	3.09099	16.82247
P_Charged	1,440.70763	33.00055
P_Fast_Charge	9.65543	0.07688
P_Grid	9,130.45935	248.66436
P_PV	2.40226	2.53062
P_Slow_Charge	9.92551	0.03614

Figure 17 – Simulation results

2. Discrete Event Simulation in Anylogic

2.1 Model description

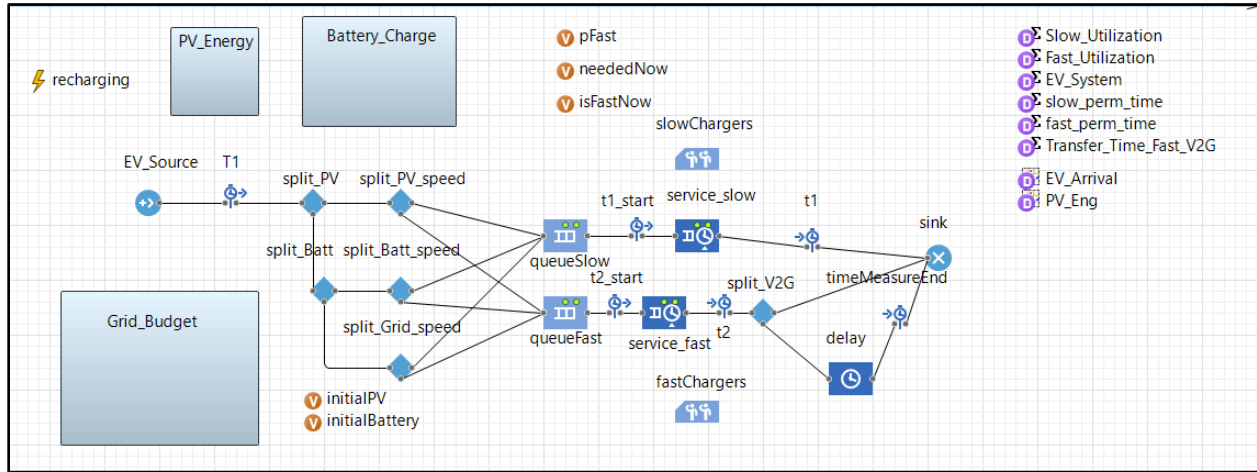


Figure 18 – DES model in Anylogic

In our AnyLogic implementation of the Smart EV Charging Hub, we have combined discrete-event blocks, resource pools, and custom logic to mirror the behavior captured in our Petri-Net model. At the top of the diagram, two stocks “PV_Energy” and “Battery_Charge” represent the instantaneous reserves of renewable and stored energy. These stocks are replenished by a single timeout event named recharging, which fires every 12 hours to add back the initial PV and battery values, thus modeling the diurnal cycle of solar generation and the overnight recovery of on-site storage. Vehicles enter the system through an EV_Source block, spaced according to a time-varying Poisson process, and immediately encounter a SelectOutput block, “split_PV,” that tests whether there is enough PV energy on hand to satisfy the vehicle’s chosen charging speed. On each arrival, we randomly assign the car as “fast” or “slow” (via uniform sampling) and compute its energy demand; if PV_Energy $\geq$ neededNow, the vehicle deducts from that stock, otherwise it falls through to the battery.

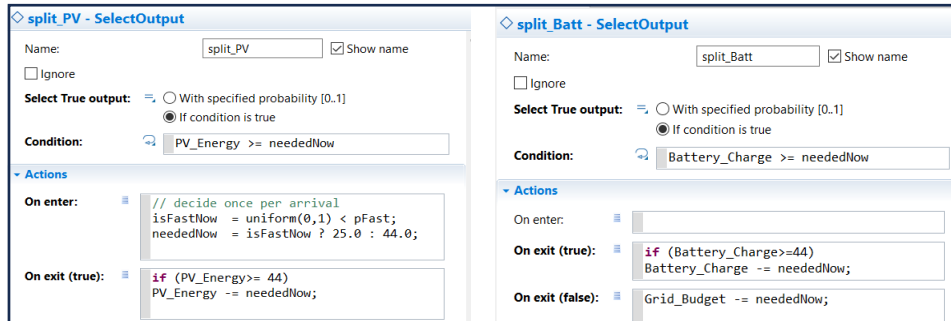


Figure 19 – Select Outputs' configuration

queueSlow - Queue	queueFast - Queue
Name: queueSlow ☒ Show name	Name: queueFast ☒ Show name ☐ Ignore
☐ Ignore	
Maximum capacity: ☐	Maximum capacity: ☐
Agent location:	Agent location:
Advanced	Advanced
Queuing: FIFO	Queuing: FIFO

Figure 20 – Queues' configuration

Below that, a second SelectOutput “split_Batt” checks whether the storage battery can supply the remaining energy; if not, the model decrements the “Grid_Budget” stock to reflect grid-purchased power. After determining both the energy source and the rate (fast or slow), vehicles join one of two FIFO queues. They then seize one unit of the corresponding ResourcePool—slowChargers or fastChargers—before entering a Service block that holds them for the appropriate charging duration (2 hours at slow, 30 minutes at fast). Upon service completion, vehicles leave via a third SelectOutput “split_V2G,” which with 80 % probability sends them to an instantaneous Sink (modeling departure) and with 20 % probability returns their unused energy to the Grid_Budget (modeling a V2G sale), then end in a global sink. Finally, a small Delay block before the sink ensures the split logic has time to execute.

service_slow - Service	service_fast - Service
Name: service_slow ☒ Show name	Name: service_fast ☒ Show name
☐ Ignore	☐ Ignore
Seize: ☒ (alternative) resource sets ☐ units of the same pool	Seize: ☒ (alternative) resource sets ☐ units of the same pool
Resource sets (alternatives): slowChargers 1	Resource sets (alternatives): fastChargers 1
Add list	Add list
Queue capacity: 10	Queue capacity: 10
Maximum queue capacity:	Maximum queue capacity:
Delay time: 2 hours	Delay time: 30 minutes
split_V2G - SelectOutput	
Name: split_V2G ☒ Show name	Name: delay ☒ Show name
☐ Ignore	☐ Ignore
Select True output: ☒ With specified probability [0..1] ☐ If condition is true	Type: ☒ Specified time ☐ Until stopDelay() is called
Probability: 0.8	Delay time: 1 hours
Actions	Maximum capacity: ☐
On enter:	Agent location:
On exit (true):	
On exit (false): Grid_Budget += neededNow;	

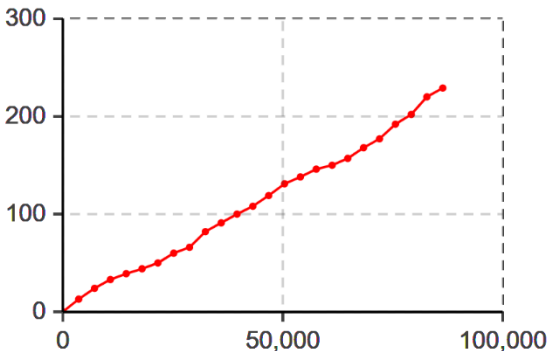
Figure 21 – Other blocks' configuration

Figure 22 – Recharging event

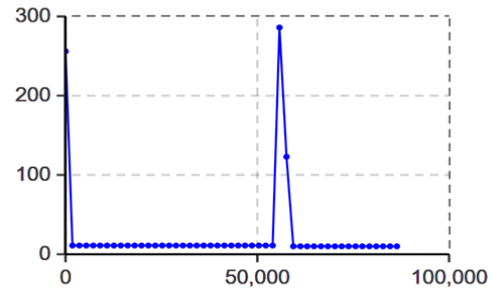
Throughout the flowchart, we capture key performance metrics using built-in statistics blocks: average queue lengths, resource utilizations, waiting times, and throughput counts. These are exposed as DataSets and Statistic elements on the right side of the canvas, so that once the Simulation experiment runs, we can automatically produce time plots, histograms, and confidence intervals directly within AnyLogic. In this way, our model not only replicates the structural behavior of the Petri net but also provides a fully parameterized, stochastic simulation environment in which we can vary arrival rates, charger counts, or battery sizes and immediately observe the impact on queue backlogs, charger utilization, and overall system throughput.

Systems properties:

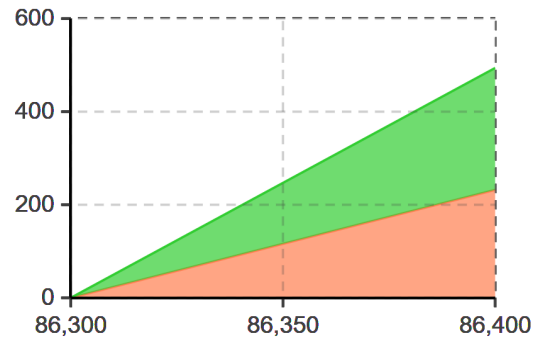
- $r_{\text{slow}} = r_{\text{fast}} = 0.5$
- $rv_{2G} = 0.2$
- $EV_Arrival = 10 \text{ [h-1]}$
- $Service_slow = 2 \text{ [h-1]}$
- $Service_fast = 0.5 \text{ [h-1]}$



● EV_Arrival



● PV Levels



● Sink

● EV_Source

2.2 Model simulation

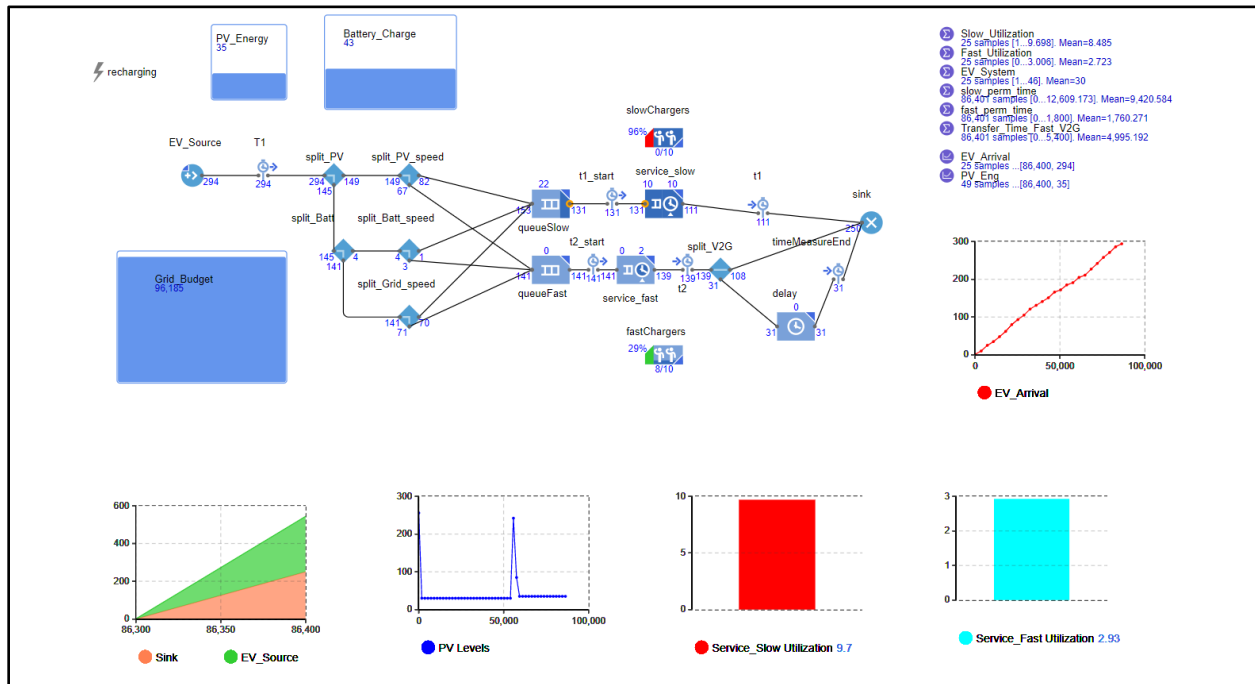


Figure 23 – Anylogic model simulation

Above we can see our fully-wired DES model in action after a 24-hour run: all of the key metrics update in real time so you can immediately see how the system behaves under your chosen parameters. First, look at the large gauge-style numbers on the left. PV_Energy shows the current state-of-charge in the solar array, and Battery_Charge underneath it shows stored energy in the site battery. We can watch both tick up by the “recharging” event at 12 h, and then deplete as incoming EV demand draws first on PV, then on battery, then on the grid. At lower left, Grid_Budget tracks cumulative draw from the grid; it climbs only when renewable sources and storage can’t fully meet instantaneous demand.

Across the top right are the statistics windows. Below the model canvas are four plots we’ve configured to show exactly the behavior you care about.

2.3 Model Results and Analysis

Let X_i be the steady-state measurement from replication ($i=1, \dots, n$). We compute:

- Sample mean:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

- Sample standard deviation:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}$$

- 95 % confidence interval (half-width)

$$\Delta = z_{0.975} \frac{s}{\sqrt{n}} \quad \text{with } z_{0.975} \approx 1.96$$

Thus our reported mean $\pm \Delta$ indicates that, under normal-approximation, $\mathbb{P}(\bar{X} - \Delta < \mu < \bar{X} + \Delta) \approx 0.95$

Below is a consolidated summary of our 24-hour / 30-replication experiment:

Metric	Mean	Deviation	Min	Max	95 % CI
EV_System (in-system vehicles)	17.04	6.07	1	26	± 2.50
Fast_Utilization (chargers)	2.65 / 10	0.59	0	3.41	± 0.24
Slow_Utilization (chargers)	7.64 / 10	2.33	1	9.30	± 0.96
Queue_Slow_Length	0.48	1.26	0	7	± 0.008
Queue_Fast_Length	0.00	0.00	0	0	± 0.000
t_1 (slow service time, h)	9 851.2	2 396.4	7 200	14 400	± 456.2
t_2 (fast service time, h)	1 800.0	0.00	1 800	1 800	± 0.00
Transfer_Time_Fast_V2G (h)	4 756.2	1 749.9	0	5 400	± 11.67
Fast_Perm_Time (h)	1 748.8	299.3	0	1 800	± 1.996
Slow_Perm_Time (h)	7 694.4	2 424.1	0	9 851.2	± 16.16
Delay_Event_Utilization	0.948	1.048	0	4	± 0.008

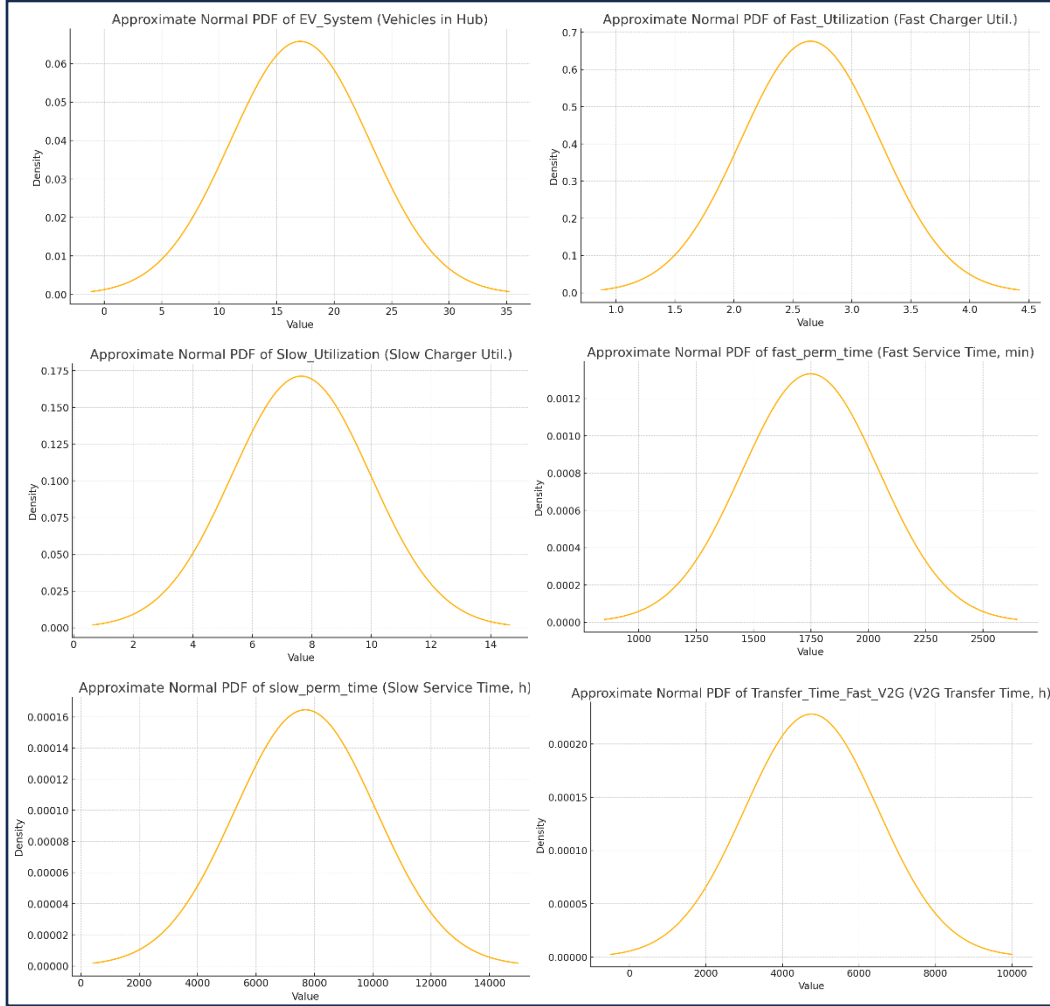
Table 3 - Results

- **Approximate Steady-State Distributions**

Below for each metric, the approximate normal density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

using the observed means μ and standard deviations σ . Even when the true service-time distributions are skewed, these normal curves help visualize central tendency and spread.



Distribution Graphs

After running thirty independent 24-hour simulations, our first striking result is the mean instantaneous backlog of electric vehicles in the system, $\langle \text{EV_System} \rangle \approx 17.0 \pm 2.5$. This tells us that, on average, seventeen vehicles are “in flight” between arrival and final exit at any moment—an immediate sign that charging capacity is nearly saturated.

